L2 ACAD A

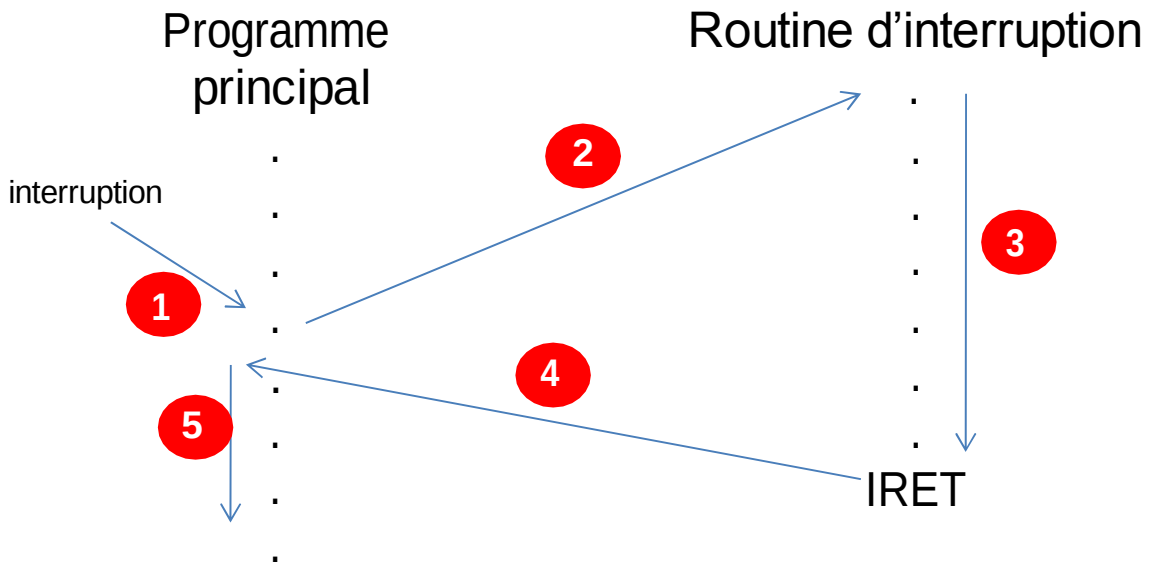
Chapitre 2

ARCHITECTURES DES ORDINATEURS II

Les interruptions

INTERRUPTION :

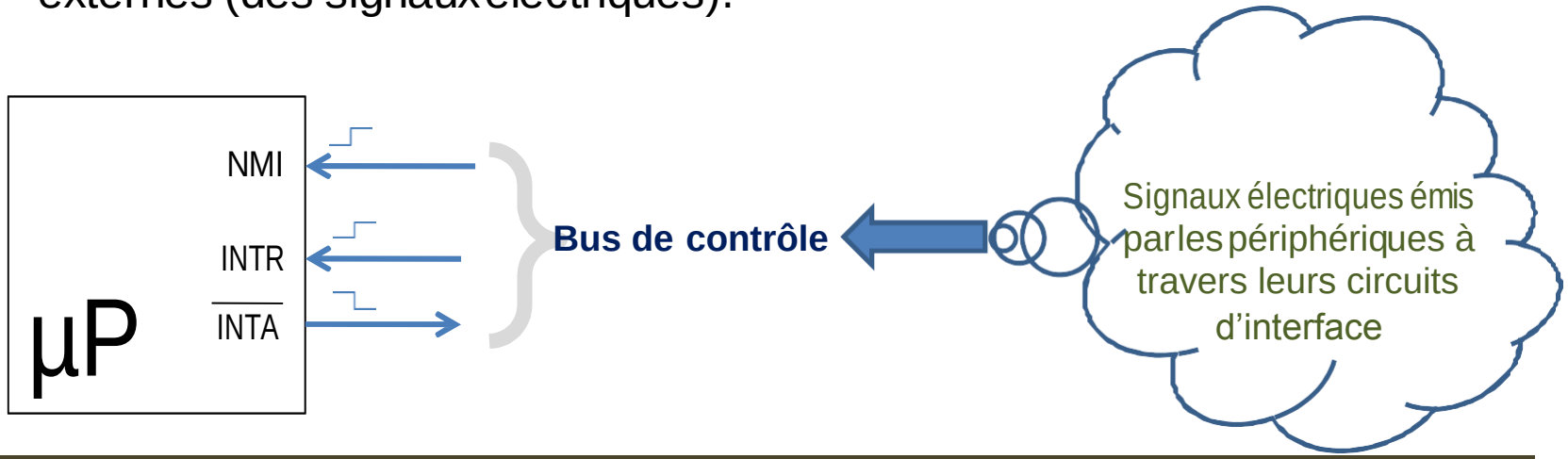
C'est un évènement (généralement) asynchrone provoquant la suspension du programme en cours d'exécution, le passage à un autre code appelé routine d'interruption (ou traitant de l'interruption) puis retour au programme suspendu. L'instruction qui permet un tel retour s'appelle IRET.



CLASSES D'INTERRUPTIONS

Il existe deux grandes classes d'interruptions: les interruption matérielles et les interruptions logicielles.

a) **Les interruptions matérielles** : elles sont provoquées par des évènements externes (des signaux électriques).



NMI : Non Maskable Interrupt
Interruption non masquable
Ne dépend pas du Flag IF

INTR : (Interrupt Request)
demande d'interruption (ou requête)
Dépend du Flag IF. IF = 0 (CLI) les requêtes sont masquées, IF = 1 (STI) les requêtes sont armées

INTA : Interrupt Acknowledgment
Acquittement de la requête (ou Accusé de réception de la requête)

b) **Les interruptions logicielles :** elles sont provoquées par des instructions d'interruption ou par des conditions logiques.

Exemple :

Instruction d'interruption : INT 21H

Condition logique : Erreur de division par zéro

NOTE:

Pour les microprocesseurs x86 de INTEL, toutes les interruptions sont **vectorisées**.

Ceci signifie que l'adresse physique (CS:IP) de chaque traitant de l'interruption est sauvegardée dans une table système appelée:

TABLE DES VECTEURS DES INTERRUPTIONS.

TABLE DES VECTEURS DES INTERRUPTIONS

Cette table système est composée de 256 vecteurs, chacun de taille 32 bits. Dans chaque vecteur est sauvegardée l'adresse physique (CS:IP) de début d'un traitant d'interruption.

ORGANISATION DE LA TABLE DES VECTEURS DES INTERRUPTIONS

- Les 5 premiers vecteurs de cette table sont réservés par INTEL à la sortie d'usine du microprocesseur.

- Certains vecteurs des 251 vecteurs restant sont réservés par le système d'exploitation.

- Le reste sont vides et peuvent être utilisés par l'utilisateur.

STRUCTURE DE LA TABLE DES VECTEURS DES INTERRUPTIONS

	B_{15} … B_0	
0000 :0000	IP_0	Vecteur 0 : IT erreur de division par zéro
	CS_0	
0000 :0004	IP_1	Vecteur 1 : IT exécution pas à pas
	CS_1	
0000 :0008	IP_2	Vecteur 2 : IT NMI
	CS_2	
0000 :000C	IP_3	Vecteur 3 : IT insertion point d'arrêt
	CS_3	
0000 :0010	IP_4	Vecteur 4 : IT overflow
	CS_4	
0000 :0014	IP_5	Vecteur 5 : IT Print Screen
	CS_5	
	. . . .	
0000 :0020	IP_8	Vecteur 8 : IT timer
	CS_8	
0000 :0024	IP_9	Vecteur 9 : IT clavier
	CS_9	
. . . . .	. . . . .	
0000 :03fc	IP_{255}	Vecteur 255
	CS_{255}	

5 vecteurs réservés à la sortie d'usine du µP

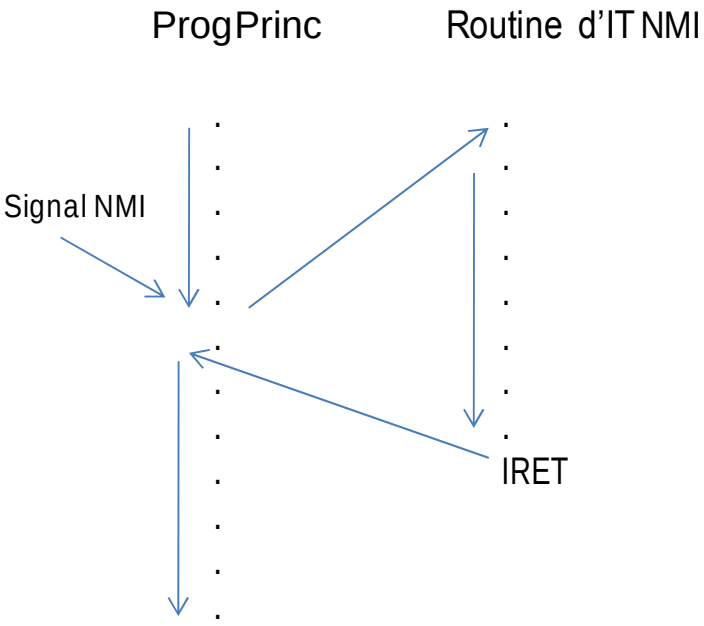
Taille de la TVI
256 x 4 Bytes
= 1024 Bytes
= 1 K Bytes

251 vecteurs, certains sont définis par le système d'exploitation et d'autres sont libres

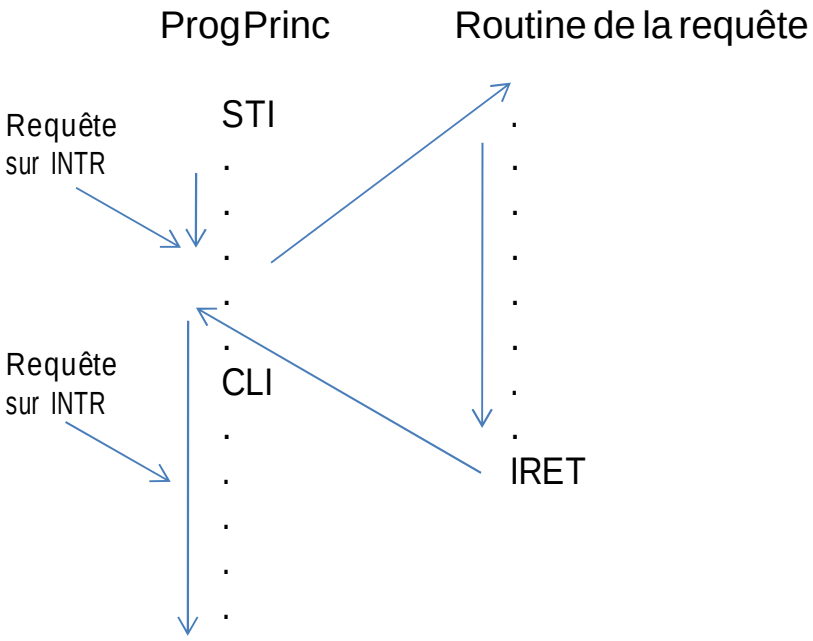
- Vecteurs réservés par intel
- Vecteurs réservés par le S.E

RECAPITULATIF SUR LES INTERRUPTIONS MATERIELLES

Cas (a) : Interruption



Cas (b) : Interruption INTR (requête ou demande d'interruption)



STI : positionner IF à 1 ; CLI : positionner IF à 0 IF : Interrupt Flag

INTERRUPTIONS LOGICIELLES :

RAPPEL:

Les interruptions logicielles sont provoquées par des instructions d'interruption ou suite à la présence d'une condition logique.

INTERRUPTION LOGICIELLE	SYNTAXE	ACTION
Appel à l'interruption logicielle n° N	INT N	(SP) ← (SP) – 2 ((SP)+1:((SP)) ← (PSW) (SP) ← (SP) – 2 ((SP)+1:((SP)) ← (CS_{retour}) (SP) ← (SP)– 2 ((SP)+1:((SP)) ← (IP_{retour}) IF = 0 , TF = 0 ($IP_{routine}$) ← (4 x N) ($CS_{routine}$) ← (4 x N +2)
Appel à l'interruption n° 3	INT	Même chose que ci-dessus avec N = 3
Appel à l'interruption overflow	INTO	Si OF = 1 alors Même chose que ci-dessus avec N = 4 Sinon aucun effet

où N représente le numéro de l'interruption logicielle invoquée , N est sur 8 bits allant entre 00H à FFH (0 à 255)

INSTRUCTION IRET

Cette instruction permet de quitter le traitant de l'interruption et le retour au programme suspendu en allant dépilant IP_{retour}, CS_{retour} et le PSW.

Instruction de retour	SYNTAXE	ACTION
Retour du traitant de l'interruption	IRET	$(IP_{retour}) \leftarrow ((SP)+1:((SP))$ $(SP) \leftarrow (SP) + 2$ $(CS_{retour}) \leftarrow ((SP)+1:((SP))$ $(SP) \leftarrow (SP) + 2$ $(PSW) \leftarrow ((SP)+1:((SP))$ $(SP) \leftarrow (SP) + 2$

APPLICATION

Soit le programme principal suivant :

```
                .
                .
0100:0350   INT 21H
                .
                .
```

Juste avant l'exécution de l'instruction INT 21H, le µP a la configuration suivante :

PSW = 03D5H
SP = 0450H
SS = 0450H

et une partie de la mémoire centrale a le contenu suivant:

0000:0082	9A
0000:0083	48
0000:0084	72
0000:0085	AC
0000:0086	D2
0000:0087	62
0000:0088	20

Juste à l'entrée du traitant de l'interruption, donner : $IP_{routine}$, $CS_{routine}$, PSW, le contenu de la pile, SS, SP.

SOLUTION :

$IP_{routine} = (4 \times N)$, N = 21H
= (4 x 21H)
= (0000:0084)
= AC72H

$CS_{routine} = (4 \times N + 2)$, N = 21H
= (4 x 21H + 2)
= (0000:0086)
= 62D2H

Rappel de la structure du PSW:
0000 ODIT SZ0A 0P0C

INT 21H force IF et TF à Zéro

Ancienne valeur du PSW avant INT 21H
PSW = 0000 0011 1101 0101B = 03D5H

PSW juste à l'entrée de la routine d'interruption
PSW = 0000 0000 1101 0101B = 00D5H

Contenu de la pile

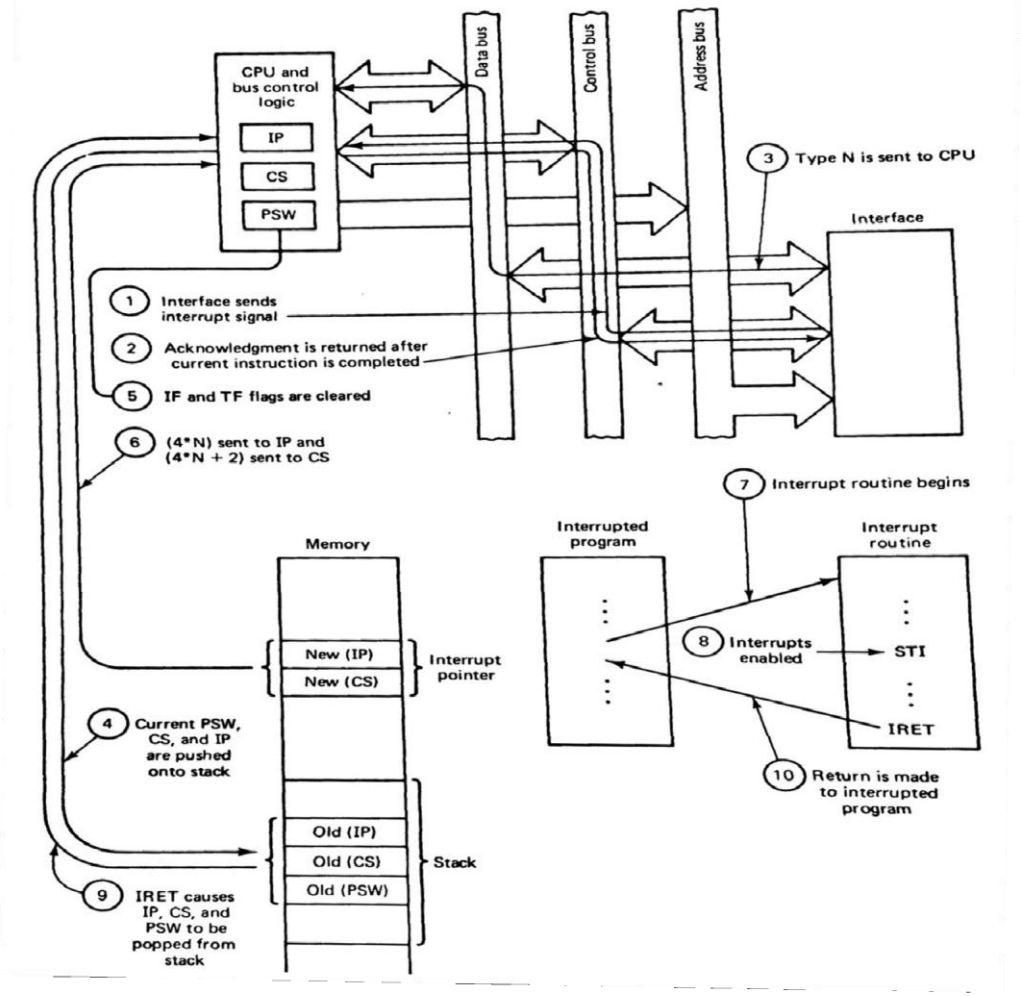
SS = 0450H
SP = 044AH

INTERRUPTIONS MATERIELLES :

RAPPEL:

Les interruptions matérielles sont provoquées par des signaux électriques externes.

CHRONOLOGIE DES EVENMENTS LORS D'UNE DEMANDE D'INTERRUPTION (REQUETE)



Un peu de pratique sur les interruptions

1. VERIFICATION DU COMPORTEMENT DU PROCESSEUR LORS D'UNE INTERRUPTION EN UTILISANT LE DEBUG

    1.1 Sauvegarde du contexte du µP avant de passer à la routine d'IT
    1.2 Préparation du µP pour passer à la routine d'IT

2. OUTILS PRATIQUES DE PROGRAMMATION DES INTERRUPTIONS

VERIFICATION DU COMPORTEMENT DU PROCESSEUR LORS D'UNE INTERRUPTION

DOSBox 0.74-2, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

Sauvegarde du contexte du µP avant de passer à la routine d'IT

```
D:\>debug
-a
073F:0100 int 48
073F:0102
-r
AX=0000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0100   NV UP EI PL NZ NA PO NC
073F:0100 CD48          INT     48
-t

AX=0000  BX=0000  CX=0000  DX=0000  SP=00F7  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=F000  IP=1060   NV UP DI PL NZ NA PO NC
F000:1060 FE38          ???     [BX+SI]                         DS:0000=CD
-d 073f:00f7
073F:00F0                       02-01 3F 07 02 F2 00 00 57          ..?.....W
073F:0100  CD 48 00 00 00 00 00 00-00 00 00 00 00 00 AE FE   .H..............
073F:0110  00 F0 93 74 00 00 B9 F4-47 0B 74 00 34 00 2E 07   ...t....G.t.4...
073F:0120  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
073F:0130  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
073F:0140  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
073F:0150  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
073F:0160  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
073F:0170  00 00 00 00 00 00 00                              .......
-_
```

DOSBox 0.74-2, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG

Préparation du µP pour passer à la routine d'IT

```
D:\>debug
-a
073F:0100 int 48
073F:0102
-r
AX=0000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=073F  IP=0100   NV UP EI PL NZ NA PO NC
073F:0100 CD48          INT     48
-t

AX=0000  BX=0000  CX=0000  DX=0000  SP=00F7  BP=0000  SI=0000  DI=0000
DS=073F  ES=073F  SS=073F  CS=F000  IP=1060   NV UP DI PL NZ NA PO NC
F000:1060 FE38          ???     [BX+SI]                         DS:0000=CD
-d 0000:0120
0000:0120  60 10 00 F0 60 10 00 F0-60 10 00 F0 00 16 00 F0   `...`...`.......
0000:0130  60 10 00 F0 60 10 00 F0-60 10 00 F0 60 10 00 F0   `...`...`...`...
0000:0140  60 10 00 F0 60 10 00 F0-60 10 00 F0 60 10 00 F0   `...`...`...`...
0000:0150  60 10 00 F0 60 10 00 F0-60 10 00 F0 60 10 00 F0   `...`...`...`...
0000:0160  60 10 00 F0 60 10 00 F0-60 10 00 F0 60 10 00 F0   `...`...`...`...
0000:0170  60 10 00 F0 60 10 00 F0-60 10 00 F0 60 10 00 F0   `...`...`...`...
0000:0180  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
0000:0190  00 00 00 00 00 00 00 00-60 10 00 F0 04 00 41 CB   ........`.....A.
-
```

PRESENTATION DE QUELQUES INTERRUPTIONS UTILES AUX TD & TP

CAS DE L'INTERRUPTION 21H :

c'est une très longue routine d'interruption parfois appelée INTERRUPTION SYSTEME , elle offre plusieurs services. Chaque service est une fonction dont le numéro doit être au préalable envoyé dans **le registre AH** avant l'appel à cette interruption.

structure de la routine d'interruption 21H appelée avec INT 21H

```
Si ah = 01H        Traitement 01H
                       Jmp fin
Si ah = 02H        Traitement 02H
                       Jmp fin
Si ah = 03H        Traitement 03H
                       Jmp fin
Si ah = 04H        Traitement 04H
                       Jmp fin
        .                  .
        .                  .
Si ah = 09H        Traitement 09H
                       Jmp fin
        .                  .
        .                  .
Si ah = 25H        Traitement 25H
                       Jmp fin
        .                  .
        .                  .
Si ah = 35H        Traitement 35H
                       Jmp fin
        .                  .
        .                  .
      fin:             IRET
```

Routine d'interruption 21H

```
; programme appelant
        .
        .
        .
        mov ah, #fonction
        int 21h
        .
        .
        .
```

QUELQUES FONCTIONS DE L'INTERRUPTION 21H

AH=01H LECTURE D'UN CARACTERE AU CLAVIER AVEC ECHO AH=02H ECRITURE D'UN CARACTERE A L'ECRAN

AH=08H LECTURE D'UN CARACTERE SANS ECHO AU CLAVIER AH=09H AFFICHAGE D'UNE CHAINE DE CARACTERS A L'ECRAN AH=0AH LECTURE D'UNE CHAINE DE CARACTERS AU CLAVIER AH=25H INSTALLATION D'UN VECTEUR D'INTERRUPTION AH=2AH LECTURE DE LA DATE SYSTEME

AH=35H LECTURE D'UN VECTEUR D'INTERRUPTION

---

1. AH=01H

LECTURE D'UN CARACTERE AU CLAVIER AVEC ECHO

ENTREE : AH=01H
RETOUR : AL=CARACTERE LU

```
; programme
        .
        .
        .
        mov ah , 1
        int 21h
        .
        .
        .
```

2. AH=02H

ECRITURE D'UN CARACTERE A L'ECRAN

ENTREE : AH=02H , DL=CARACTERE A

ECRIRE RETOUR : RIEN

3. AH=08H

LECTURE D'UN CARACTERE SANS ECHO AU CLAVIER

ENTREE : AH=08H

RETOUR : AL=CARACTERE LU

4. AH=09H

AFFICHAGE D'UNE CHAINE DE CARACTERES A L'ECRAN

ENTREE : AH=09H

DS:DX ADRESSE DU 1er CARACTERE DE LA CHAINE A AFFICHER

(la chaîne de caractères doit se terminer par "$")

RETOUR : RIEN

5. AH=0AH

LECTURE D'UNE CHAINE DE CARACTERS AU CLAVIER

ENTREE : AH=0AH ,

DS:DX ADRESSE DU PREMIER OCTET DE LA CHAINE

(la case mémoire DS:DX doit être initialisée à la taille max du buffer de réception)

DS :DX+1 reçoit à la fin de la lecture le nombre de caractères réellement reçu,

A partir de DS :DX+2 la chaîne reçue

RETOUR : RIEN

```
; programme
data        segment
msg     db  'Bonjour Archi2$'
data        ends
code        segment
assume cs: code , ds, data
start: mov ax, data
       mov ds, ax.
       .
       .
       mov ax, seg msg
       mov ds, ax
       mov dx , offset msg
       mov ah , 9
       int 21h
       .
       .
code        ends
            end start
```

6. AH=2AH
LECTURE DE LA DATE SYSTEME

ENTREE : AH=2AH
RETOUR : CX=ANNEE (0=1980..119=2099)
DH=MOIS
(1=JANVIER,2=FEVRIER..)
DL=JOUR (1...31)

7. AH=25H
INSTALLATION D'UN VECTEUR D'INTERRUPTION

ENTREE : AH=25H
AL= NUMERO D'INTERRUPTION
DS:DX ADRESSE DE LA PREMIERE INSTRUCTION DE LA ROUTINE D'INTERRUPTION
RETOUR : RIEN

8. AH=35H
LECTURE D'UN VECTEUR D'INTERRUPTION

ENTREE : AH=35H
AL= NUMERO D'INTERRUPTION
RETOUR : ES:BX ADRESSE DE DEBUT DE LA ROUTINE D'INTERRUPTION

POSITIONNER LE CURSEUR :

```
mov ah,2 ;numéro de
fonction mov bh,0
mov dh,lig ;ligne de
l'écran mov dl,col
;colonne de l'écran int
10h
```

EFFACER ECRAN :

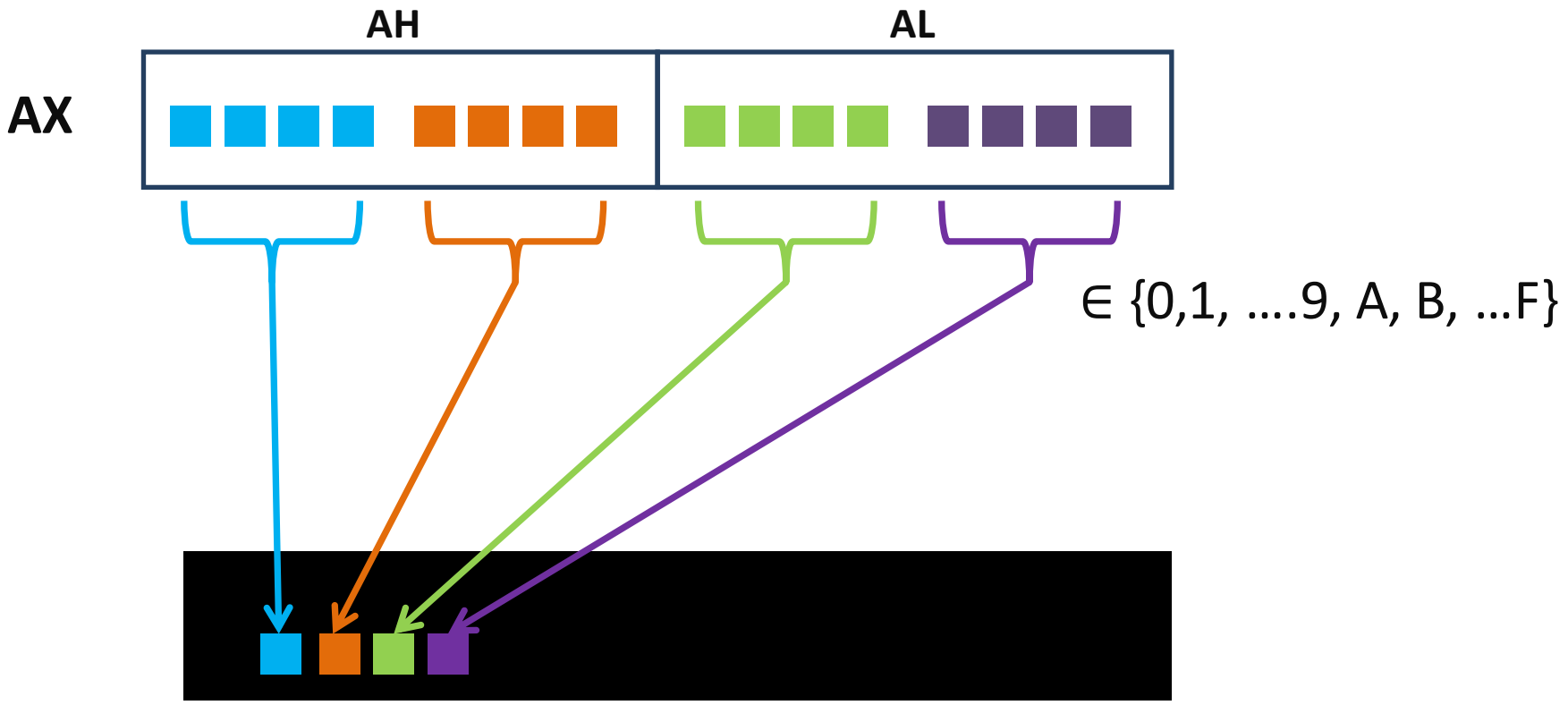
```
mov ax , 3
int 10h
```

Exemples d'applications : Affichage d'un nombre sur écran en utilisant les fonctions d'interruptions

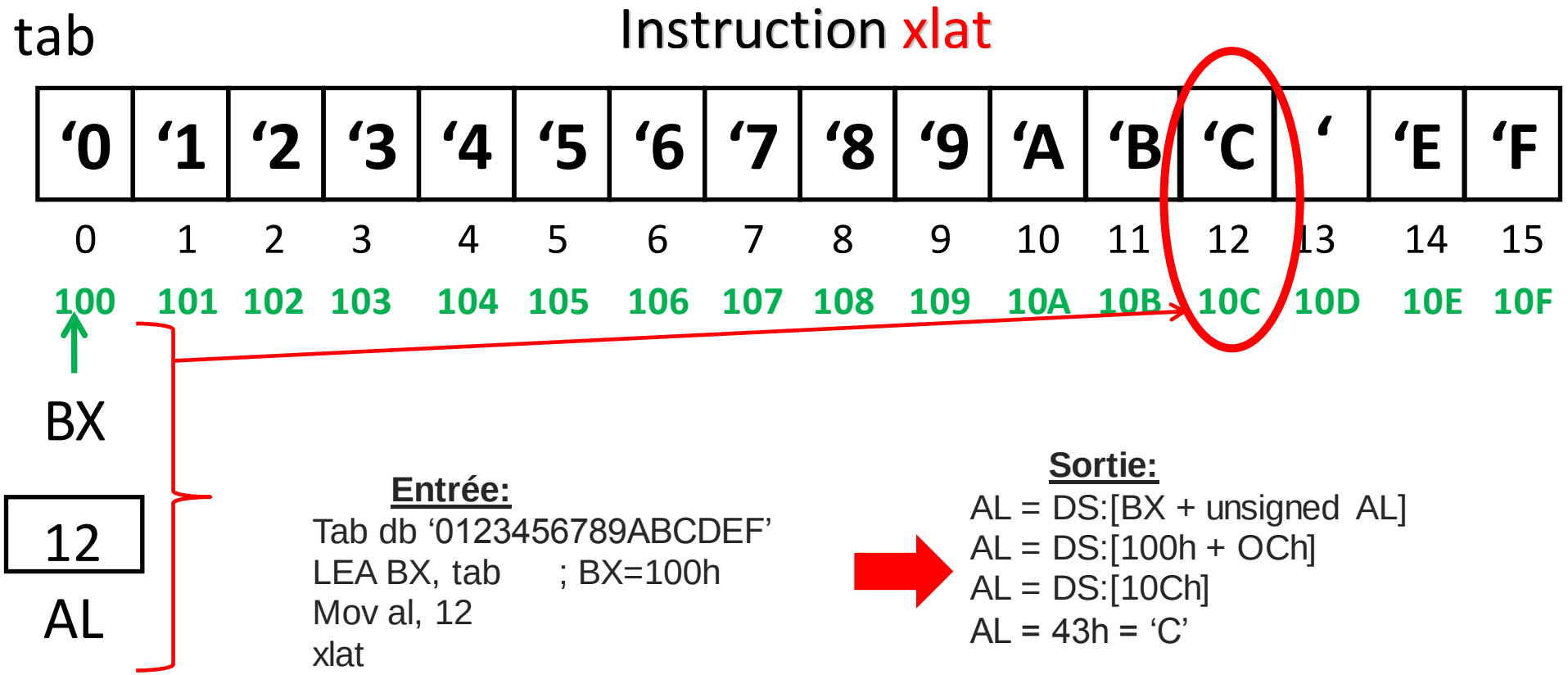
Application 1 : Affichage d'un nombre en format hexadécimal en utilisant la fonction 2h/21h ou la fonction 0Eh/10h

Principe de la solution : La méthode utilisée pour convertir une donnée présentée en format binaire en une donnée hexadécimale est la méthode de regroupement par quatre bits qui consiste à regrouper les bits adjacents quatre par quatre et de les évaluer en hexadécimal par une valeur comprise entre 0 et F (15 étant la valeur max pour un groupe de 4 bits à 1).
Une fois le regroupement par 4 bits est fait, la prochaine étape consiste à remplacer chaque valeur d'un groupe par le code ascii du caractère hexadécimal correspondant et d'utiliser la fonction 2H de l'interruption système 21H ou bien la fonction 0Eh de l'interruption BIOS 10H qui permettent l'affichage d'un caractère sur écran.

Le sens d'affichage se fait de gauche à droite comme présenté si dessous :



La deuxième étape de la méthode peut se faire en créant une table de correspondance des valeurs 0 à 15 respectivement avec les codes ascii des caractères correspondants ‘0’ à ‘F’.
L’instruction assembleur permettant de réaliser cette correspondance est XLAT :



Entrée:

Tab db ‘0123456789ABCDEF’
LEA BX, tab ; BX=100h
Mov al, 12
xlat

Sortie:

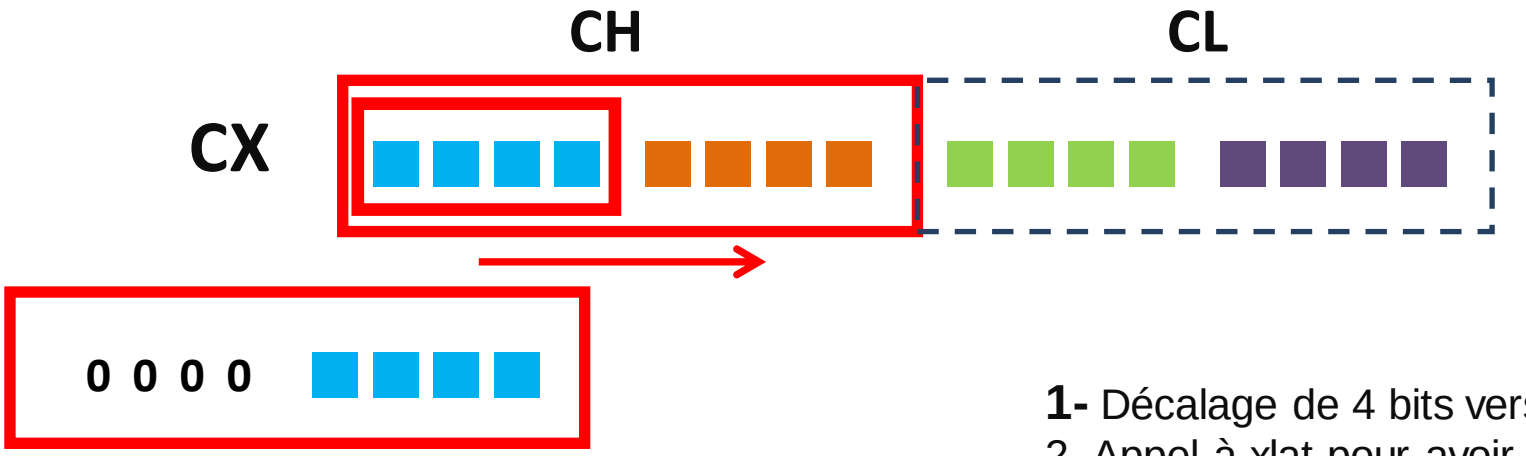
AL = DS:[BX + unsigned AL]
AL = DS:[100h + OCh]
AL = DS:[10Ch]
AL = 43h = ‘C’

Translate byte from table.
Copy value of memory byte at DS:[BX + unsigned AL] to AL register.

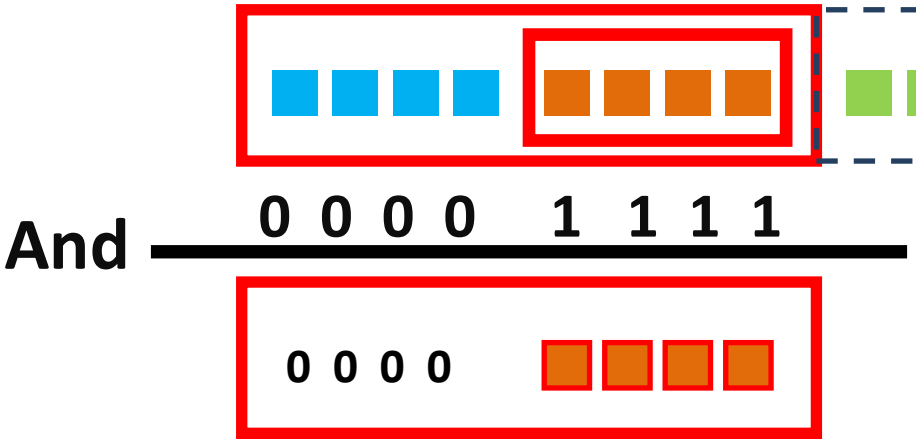
Algorithm:

AL = DS:[BX + unsigned AL]

Les différentes phases de la technique de regroupement par 4 bits d'une donnée 16 bits, exemple CX :



1- Décalage de 4 bits vers la droite
2- Appel à xlat pour avoir le code ascii du caractère
3- Appel à la fonction d'interruption pour afficher le caractère



1- And logique avec la valeur 0Fh
2- Appel à xlat pour avoir le code ascii du caractère
3- Appel à la fonction d'interruption pour afficher le caractère

- Refaire les mêmes étapes pour cl

Code assembleur affichage du contenu de CX en héxadécimal :

```
; Procédure qui fait correspondre à une valeur le code ascii du caractère associé et affiche le caractère en utilisant la fonction 0Eh/21h
afficheHexa PROC NEAR
xlat
mov ah, 0Eh
int 21h
RET
afficheHexa ENDP
….
Start:
…..
Mov al, ch
Shr al, 4 ; Le groupe de 4 bits de poid fort de CH
Call near ptr afficheHexa
Mov al, ch
And al, 0Fh ; Le groupe de 4 bits de poid faible de CH
Call near ptr afficheHexa
; Refaire les memes instructions pour afficher le contenu de CL en héxadécimal
```

La fonction 0Eh/21h peut être remplacée par la fonction 2h/21h à condition de mettre le code ascii du caractère à afficher dans dl au lieu de al.

Application 2 : Affichage décimal d'un nombre en utilisant la fonction 2h/21h ou la fonction 9h/21h

Principe de la solution : la méthode consiste à séparer les chiffres qui constituent le nombre et les afficher séparément, les chiffres constituant un nombre donné peuvent être obtenu par une succession de division par la base (10) et la récupération des restes de division de chaque. L'arrêt du processus de division se fait suite à l'obtention d'un reste de division nul.
Les chiffres doivent être affichés dans le sens inverse de leur obtention, pour cela, deux solutions sont possibles :

La première est de sauvegarder les restes de division dans une pile, à la fin les chiffres (codes ascii associés aux chiffres) vont être restaurés et affichés en sens inverse (du dernier au premier) caractère par caractère en appelant la fonction 2h/21h.

La deuxième solution consiste à stocker les restes de division dans une chaine de caractère après avoir converti chaque chiffre en caractère associé et ce à partir de la fin de la chine de caractère, ensuite afficher la chaine en utilisant la fonction 9h/21h.
Pour avoir le code ascii d'un nombre compris entre 0 et 9 sachant que le code ascii de 0 est la valeur 30h, il suffit de rajouter 30h à chaque valeur.

0	1	2	3	4	5	6	7	8	9
30H	31H	32H	33H	34H	35H	36H	37H	38H	39H

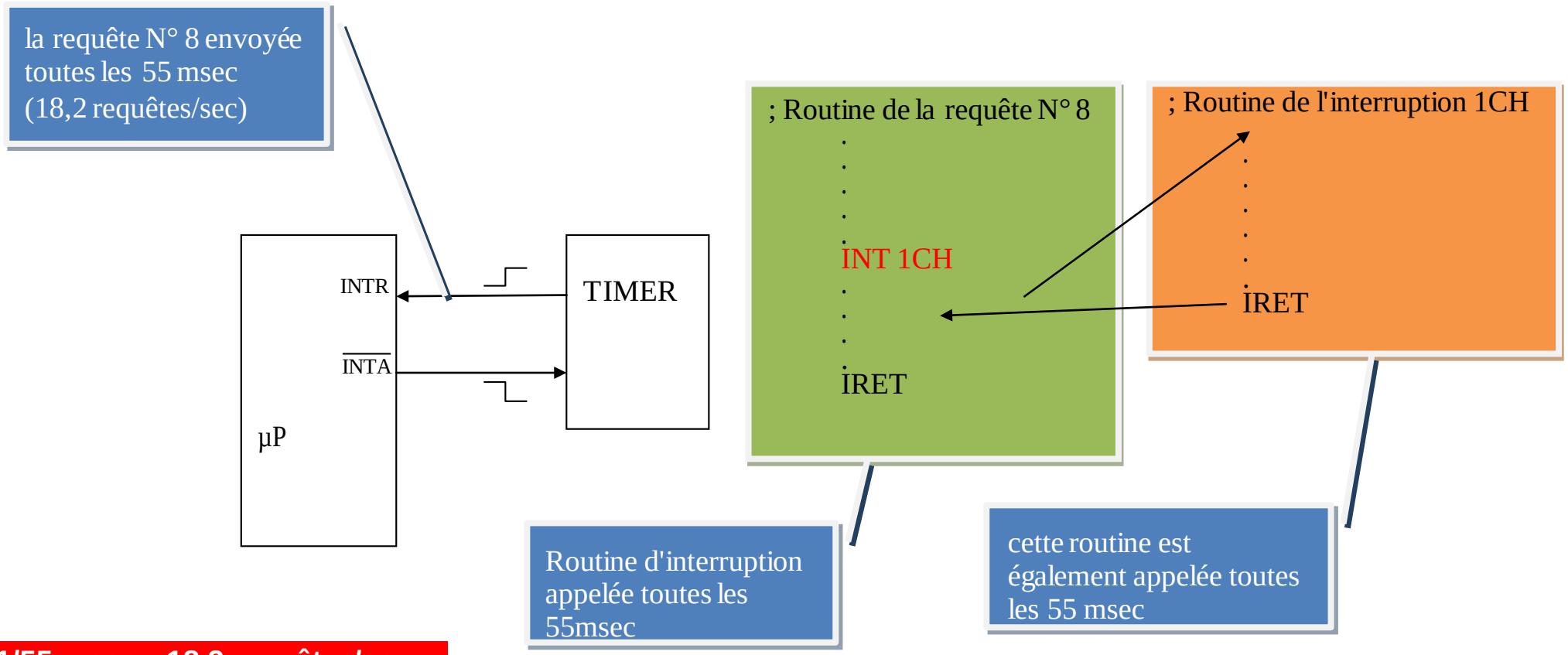
Code assembleur affichage du contenu de AX en décimal en utilisant la fonction 9h/21h :

Chaine db 10 dup (?) ***; réserver un espace mémoire d'une taille max***
……..
Mov si, offset chaine

```
Add si, length chaine
Dec si
mov byte ptr[si], '$' ; écrire le caractère $ en fin de chaine pour pouvoir l'afficher par la fonction 9h/21h
mov bl , 10
etiq : dec si
        xor ah, ah
        div bl ; après la division ah va contenir le reste de division et al contient le résultat de la division
        add ah, 30h ; rajouter au reste de division 30h pour avoir le code ascii du caractère
        mov [si], ah ; écrire le caractère dans a chaine en décrémentant la position de si
        cmp al, 0 ; répéter jusqu'à la rencontre d'un résultat de division nul
        jne etiq

mov dx, si ;
mov ah, 9h ;appeler la fonction 9h de l'interruption système 21h pour afficher la chaine
int 21h
```

INTERRUPTION 1CH :



1/55 msec = 18,2 requêtes/sec

Le code de la routine d'interruption 1CH est sans aucun intérêt et donc prévu pour un éventuel déroutement afin d'implémenter des quantums de temps indépendamment du processeur

DEROUTEMENT D'UNE INTERRUPTION

Le déroutement de l'interruption numéro N consiste à conserver le numéro de l'interruption N et donc la conservation de l'adresse du vecteur N i.e 4 x N et le changement du contenu de ce vecteur i.e (4 x N) et (4 x N + 2). En d'autres termes ceci consiste à conserver le numéro de l'interruption et changer son traitement.

```
; Programme  principal
.
        .
        .
        .
        int 48h
        .
        .
        .
    mov ax , seg new
    mov ds , ax
    mov dx , offset new
    mov al , 48h
    mov ah , 25h
    int 21h

    int 48H
        .
        .
```

```
; Ancienne  Routine  d'IT N° 48H
        .
        .
    add ax , bx
        .
        .
      IRET
```

Ancien traitement

(a) avant déroutement

```
; Nouvelle  Routine  d'IT N° 48H
  new:  .
        .
    sub cx , dx
        .
        .
      IRET
```

Nouveau traitement

(b) après déroutement

Rappel:

AH=25H , INT 21H
INSTALLATION D'UN VECTEUR D'INTERRUPTION

ENTREE____:
AH=25H
DS:DX ADRESSE DE LA ROUTINE D'INTERRUPTION AL= NUMERO DE L'INTERRUPTION

RETOUR____: RIEN

Cours réalisé par :
Mr Ayache, Mme Bouchene